
Large Language Model Inference Systems

From one request to a production serving stack

Shuhao Zhang

Huazhong University of Science and Technology

Three-Hour English Teaching Edition
2026

A concrete story: one chatbot question

User sees

- Type a question.
- Wait for the first word.
- Watch tokens stream out.
- Abandon the task if the service feels slow.

System does

- Tokenize and validate the request.
- Queue it with other users' requests.
- Run prefill to create KV cache.
- Run decode for many iterations.
- Stream output and recycle memory.

LLM serving is an online stateful system.

Three-hour course roadmap

Session 1: model to request

- Tokens, attention, and Transformer blocks
- Autoregressive generation
- Prefill, decode, and service metrics

Session 2: serving engine

- Request and iteration scheduling
- Continuous batching and KV paging
- State ownership, reuse, and pressure

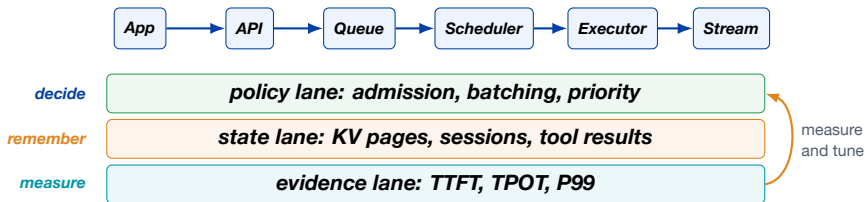
Session 3: systems evidence

- RAG, MoE, agents, and accelerators
- Heterogeneous execution
- Benchmarking and open engineering

Guiding thread

Follow the tensor, follow the request, follow the state, and verify the evidence.

The big picture



- Many optimizations do not happen inside a single kernel.
- They happen at control points: queueing, batching, caching, memory movement, and validation.

Part I. From Tokens to Autoregressive Inference

Build the model-side foundation before opening the serving engine

Start with one concrete prompt

Text entered by the user

“I love AI.”

1. The tokenizer splits text into tokens.
2. Embedding turns each token ID into a vector.
3. Transformer blocks mix information across positions and channels.
4. The language-model head scores the next token.

An inference engine serves this computation repeatedly and concurrently.

Token count is not character count

Teaching example

Suppose the tokenizer returns

[I, love, AI, .]

The sequence length is therefore
 $S = 4$.

Do not confuse

- Text characters
- Token IDs
- Sequence length S
- Vector width d_{model}

If $d_{\text{model}} = 8$, the four token vectors form a 4×8 matrix.

What question does attention answer?

For every query position

Which earlier positions are useful, and how much information should be taken from each one?

- Q : what the current position is looking for.
- K : what each position offers for matching.
- V : the information that will actually be mixed.

Match with QK^T ; mix the V vectors.

Worked attention example: define one row

We compute only the row for token 3: AI

Use one head with $d_{\text{head}} = 2$:

$$q_{\text{AI}} = [1, 1]$$

$$k_{\text{I}} = [1, 0], \quad k_{\text{love}} = [0, 1], \quad k_{\text{AI}} = [1, 1], \quad k_{\text{.}} = [0, 0].$$

- Four query positions and four key positions create a 4×4 score matrix.
- This page follows only its third row; the other rows use the same procedure.

Step 1: compute the four attention scores

$$\frac{q_{AI}K^{\top}}{\sqrt{2}} = \frac{[1, 1, 2, 0]}{\sqrt{2}} \approx [0.707, 0.707, 1.414, 0].$$

1. Dot product with I: $1 \cdot 1 + 1 \cdot 0 = 1$.
2. Dot product with love: 1.
3. Dot product with AI: 2.
4. Dot product with .: 0.

The scale factor $\sqrt{d_{\text{head}}}$ keeps large dimensions from producing overly sharp scores.

Step 2: the causal mask blocks the future

Token 3 may read positions 1–3, but not position 4

$$[0.707, 0.707, 1.414, 0] \longrightarrow [0.707, 0.707, 1.414, -\infty].$$

- The fourth softmax weight becomes zero.
- The model cannot use a future token to predict an earlier token.
- All known query rows in the same layer can still be computed in parallel.

Step 3: softmax turns scores into weights

$$\text{softmax}([0.707, 0.707, 1.414, -\infty]) \approx [0.248, 0.248, 0.503, 0].$$

- Every weight is non-negative.
- The weights sum to approximately one.
- These are position-mixing weights for one layer and one head.

They are not next-token probabilities over the vocabulary.

Step 4: attention returns a weighted sum of values

Let

$$v_I = [1, 0], \quad v_{\text{love}} = [0, 2], \quad v_{\text{AI}} = [2, 1].$$

Then

$$o_{\text{AI}} = 0.248[1, 0] + 0.248[0, 2] + 0.503[2, 1] \approx [1.254, 0.999].$$

Plain-language meaning

The representation of AI now contains a weighted mixture of information from visible positions.

Multi-head attention repeats the same idea

- Each head has its own Q , K , and V projections.
- Different heads can learn different position relationships.
- Head outputs are concatenated and projected back to d_{model} .

Shape example

With $B = 2$, $S = 4$, $d_{\text{model}} = 8$, and two heads:

$$X : 2 \times 4 \times 8 \rightarrow Q, K, V : 2 \times 2 \times 4 \times 4 \rightarrow O : 2 \times 4 \times 8.$$

A Transformer block preserves the main shape

1. Normalize the $B \times S \times d_{\text{model}}$ tensor.
2. Attention mixes information along the sequence dimension.
3. Residual addition returns to the main path.
4. MLP expands and contracts the channel dimension.
5. A second residual addition completes the block.

Shape invariant

For $X : 2 \times 4 \times 8$, the block output is still $2 \times 4 \times 8$.

The model selects one next token per sequence

After the final Transformer block

Take the final position from each sequence, map its d_{model} -dimensional vector to $|V|$ logits, then sample or select one token.

- Logits are unnormalized vocabulary scores.
- Temperature, top- k , and top- p change token selection.
- The selected token is appended to the sequence.
- The next forward step can begin only after this choice.

Autoregressive generation is a loop

1. Process the known prompt.
2. Produce logits for the next position.
3. Select one token.
4. Append it to the sequence.
5. Repeat until a stop condition is reached.

System consequence

Prompt positions are known together, but output tokens are causally dependent across time. This creates the prefill/decode split.

Checkpoint: explain the model side without jargon

1. Why does four tokens produce a 4×4 attention score matrix?
2. What does the causal mask remove?
3. Why is attention output a weighted sum of V , not a vocabulary probability?
4. Why does generation need a repeated loop?

Suggested classroom format: two minutes alone, two minutes with a partner, then one shared explanation.

Part II. Request Lifecycle and Service Quality

Connect autoregressive generation to user-visible serving behavior

From model capability to service capability

Models changed

- Long context
- MoE
- Multimodal input
- Tool-using agents

Services changed

- Chat APIs
- Enterprise assistants
- Scientific agents
- Multi-tenant platforms

Metrics changed

- First-token delay
- Tail latency
- Cost per request
- SLO satisfaction

Takeaway

A good model can still become a bad product if the serving system is slow, unstable, or too expensive.

Throughput alone is incomplete

Server A

- High average throughput.
- Large batches.
- Some users wait a long time before the first token.

Server B

- Slightly lower throughput.
- Smaller waiting time.
- More requests meet the latency target.

Better service depends on the objective: throughput, first-token latency, tail latency, or SLO compliance.

Training systems vs. inference systems

Dimension	Training	Inference serving
Workload	Large stable batches	Online arrivals, mixed request lengths
Objective	Time-to-train, scaling efficiency	TTFT, TPOT, P99, cost, SLO
State	Parameters, gradients, optimizer states	KV cache, queues, sessions, logs
Control point	Step / micro-batch	Request / iteration / token
Failure mode	Slow convergence, low utilization	Jitter, head-of-line blocking, cache pressure

Working view

Inference serving behaves like a **stateful online service**, not a static model call.

Four recurring checks

1. **How are requests organized?** Queueing, batching, priorities, and SLOs.
2. **How is state managed?** KV allocation, reuse, migration, and eviction.
3. **Which execution path is used?** Kernels, graph replay, backends, and fallbacks.
4. **What evidence supports the result?** Workload, baseline, metadata, and metrics.

Request

State

Execution

Evidence

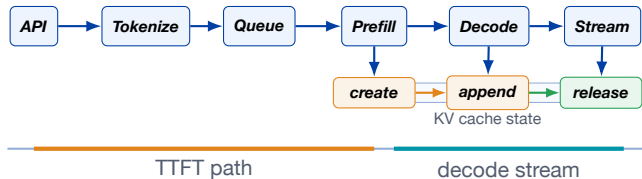
The request lifecycle

1. API receives a prompt and generation parameters.
2. Tokenizer converts text into token IDs.
3. Scheduler admits the request into a waiting queue.
4. Prefill processes the prompt and creates the first KV cache.
5. Decode repeatedly generates new tokens.
6. The server streams output and releases resources.

Why this view matters

A clear performance claim names the stage it changes and the cost it moves.

Lifecycle diagram



- TTFT is dominated by queueing and prefill.
- Later token latency is dominated by decode scheduling and KV residency.

Prefill and decode are different workloads

Prefill

- Reads the whole prompt.
- More compute intensive.
- Determines first-token latency.
- Becomes expensive with long context and RAG.

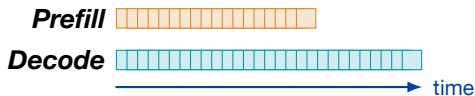
Decode

- Generates one or a few tokens per iteration.
- Lighter per step, but repeated many times.
- Relies on KV cache staying alive.
- Sensitive to scheduling jitter.

They expose different bottlenecks and need different optimization paths.

Example request: “Explain transformer attention”

- Prompt: 600 tokens.
- Output: 120 tokens.
- Prefill: one relatively heavy pass.
- Decode: 120 small iterations.



Observation

A user feels both the wait before the first token and the smoothness of the stream afterwards.

Service metrics

Metric	Meaning
<i>TTFT</i>	Time to first token. Strongly affected by queueing and prefill.
<i>TPOT / ITL</i>	Time per output token. Indicates decode steady-state behavior.
<i>Throughput</i>	Tokens/sec or requests/sec. Useful but can hide tail latency.
<i>Goodput</i>	Useful throughput under a latency target. Often better for online services.
<i>P95 / P99</i>	Tail latency. Important for multi-tenant and bursty workloads.

Metric set

A serving result is clearest when throughput, latency, tail behavior, and configuration are reported together.

Where latency is spent

Before the first token

- Queueing and admission.
- Tokenization and request validation.
- Prefill over the input context.
- KV allocation and graph path selection.

After the first token

- Decode iterations.
- Batch reshaping.
- KV page lookup and residency.
- Stream backpressure and tail behavior.

The same request contributes to different bottlenecks at different times.

10-Minute Break

Session 2 resumes with scheduling, continuous batching, and KV state.

Before the next session

Keep three objects in mind: the candidate requests, the token budget, and the free KV blocks.

Part III. Scheduling and State Management

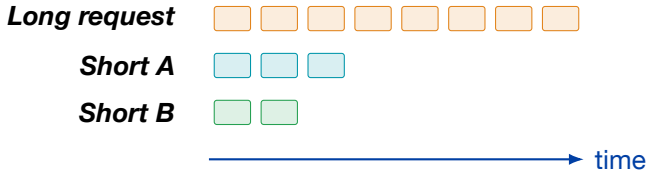
Continuous batching, KV cache, and stateful execution

Scheduling tension

Online serving must continuously organize requests with very different lengths into efficient batches.

- Bigger batches improve device utilization.
- Bigger batches may also make new users wait longer.
- Long requests keep KV state alive for many iterations.
- Short requests need low waiting time even when long requests are active.

Head-of-line blocking in mixed workloads



- If the system waits for the whole request to finish, short requests may wait unnecessarily.
- Average throughput may look acceptable while TTFT and P99 degrade.

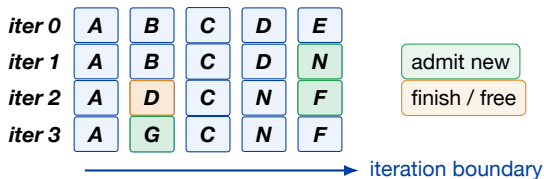
Request-level vs. iteration-level scheduling

Granularity	Benefit	Risk
Request	Simple implementation	Poor ability to insert new requests during generation
Iteration	Rebuild batch after each decode round	Needs careful state tracking and KV management
Token	Maximum flexibility	High metadata and synchronization overhead

Orca-style intuition

Move the scheduling decision from the request boundary to the generation-iteration boundary.

Continuous batching



- New requests can enter when old requests finish.
- The price is that every active request needs its own state tracked across iterations.

Worked scheduler example: two budgets, three requests

This iteration has two hard limits

Token budget = 8; free KV blocks = 3.

Request	Tokens needed	KV blocks needed
A	4	2
B	3	1
C	5	3

Question: which pairs fit both budgets?

Check every candidate pair

1. $A + B = (7 \text{ tokens}, 3 \text{ blocks})$: feasible.
2. $A + C = (9 \text{ tokens}, 5 \text{ blocks})$: exceeds both budgets.
3. $B + C = (8 \text{ tokens}, 4 \text{ blocks})$: token budget fits, KV budget does not.

Result

$A + B$ is the only feasible pair. Hard constraints define the feasible set; priority and fairness choose among feasible options.

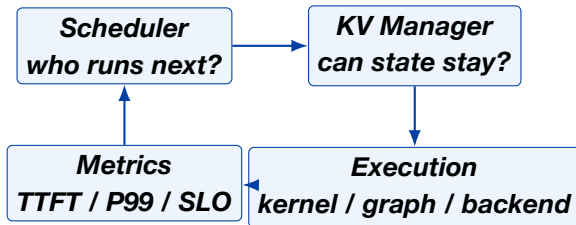
Feasible does not automatically mean fair

Follow-up scenario

Request C has waited for two seconds, while A and B have just arrived.

- Repeatedly choosing short requests may improve immediate throughput.
- The long-waiting request may starve.
- Aging, reservations, or SLO-aware priority can bound waiting time.
- The evaluation must report throughput, TTFT distribution, TPOT, and maximum wait.

Scheduling is not isolated



Coupling point

Scheduling decisions, KV residency, execution path, and service metrics form one feedback loop.

Why KV cache is central

- KV grows with context length and concurrency.
- It must stay resident across many decode iterations.
- It limits effective batch size.
- It turns every active request into a stateful object.
- Once prefix reuse is enabled, KV becomes a shared resource.

KV cache is not an implementation detail. It is a first-class system resource.

Worked KV example: bytes per token

Model configuration

$L = 32$ layers, $H_{kv} = 8$ KV heads, $d_{head} = 128$, FP16 = 2 bytes per value.

$$\begin{aligned}\text{KV bytes/token} &= 2 \times L \times H_{kv} \times d_{head} \times \text{bytes} \\ &= 2 \times 32 \times 8 \times 128 \times 2 = 131,072 \text{ bytes} = 128 \text{ KiB}.\end{aligned}$$

- The first factor of two means one K and one V .
- 100 new tokens add approximately 12.5 MiB per request.

Worked paging example: 40 tokens, block size 16

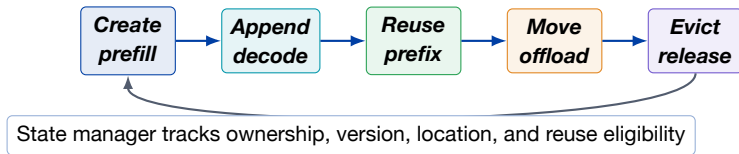
1. Tokens 1–16 fill logical block 0.
2. Tokens 17–32 fill logical block 1.
3. Tokens 33–40 require logical block 2.

$$\text{blocks} = \left\lceil \frac{40}{16} \right\rceil = 3, \quad \text{unused slots} = 3 \times 16 - 40 = 8.$$

Interpretation

Paging avoids a contiguous 40-token allocation, but capacity still grows in block-sized units.

KV cache lifecycle



Design point

Each optimization changes one lifecycle step and may shift cost elsewhere.

Request state and physical KV state are different

Example: request R17

WAITING → RUNNING → PREEMPTED → RUNNING → FINISHED.

- Logical state records tokens, phase, and progress.
- Physical state records devices, blocks, addresses, and in-flight operations.
- After preemption, the same logical tokens may resume on different physical blocks.
- Release is safe only after ownership, references, and asynchronous use are resolved.

Shared prefix example: reference count prevents early reuse

Initial state

Requests R17 and R18 share prefix block P3, so $\text{refcount}(P3) = 2$.

1. R17 is cancelled: the count drops from 2 to 1.
2. P3 cannot return to the free list because R18 still reads it.
3. R18 finishes: the count drops from 1 to 0.
4. After in-flight device reads complete, P3 may be reclaimed.

Reference count handles sharing; a device event or fence handles asynchronous completion.

Why contiguous allocation breaks down

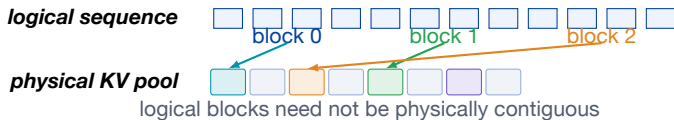
If KV must be contiguous

- Dynamic lengths create fragmentation.
- Reserving too much wastes memory.
- Reserving too little causes failures.

Paged idea

- Split KV into blocks.
- Map logical sequence to physical pages.
- Let the scheduler manage more dynamic requests.

Paged KV: a visual analogy

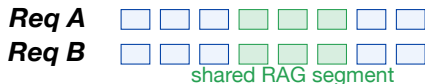


System effect

PagedAttention-style management reduces memory waste, supports dynamic batching, and creates a handle for reuse, migration, and multi-level KV storage.

Sharing KV: a small RAG example

- User A asks about a course policy.
- User B asks a different question about the same policy document.
- Both prompts include the same retrieved paragraph.



Opportunity: reuse the expensive state for the shared segment.

But sharing has a contract

Version, parameters, permissions, and semantic boundaries must match.

State decisions made by the runtime

Decision	Local benefit	System cost
Keep	Avoid recomputation	Occupies memory and limits new batches
Reuse	Save prefill work	Requires matching, validation, and ownership checks
Move	Release device memory	Adds transfer latency and coordination
Evict	Admit urgent work	May trigger recomputation later

Scheduling view

The scheduler is no longer choosing only which request runs next; it also chooses which state remains valuable.

Checkpoint: make one scheduling decision

Given

Token budget = 10, free blocks = 4. Request D needs (6, 2), E needs (4, 2), and F needs (3, 3).

1. List all feasible request pairs.
2. Choose one pair if F has the oldest deadline.
3. State which metric could become worse because of your choice.
4. Name the event fields needed to explain the decision later.

A complete answer separates feasibility, policy, expected effect, and evidence.

Part IV. Modern Workloads and Heterogeneous Execution

Long context, RAG, MoE, agents, and accelerator dynamics

Scenario

The GPU/NPU memory is almost full. A new high-priority request arrives.

Option 1

Keep all KV, reject or delay the new request.

Option 2

Preempt one request and keep its KV elsewhere.

Option 3

Drop KV and recompute later.

Scheduling now includes a state-utility decision.

New forms of intermediate state

State form	Where it comes from	System challenge
KV blocks	Long context and continuous batching	Memory pressure, fragmentation, eviction
Prefix / segment	System prompts, RAG, templates	Matching, consistency, reuse cost
Draft state	Speculative decoding	Branching, rollback, commit
Agent state	Tool calls and multi-turn memory	Cross-step propagation, isolation

Research trend

Intermediate state is becoming a managed resource, not a temporary tensor.

Long context and RAG

- Longer prompts amplify prefill cost.
- RAG makes input length and overlap more variable.
- Similar documents create reuse opportunities.
- The benefit depends on matching cost, memory pressure, and latency targets.

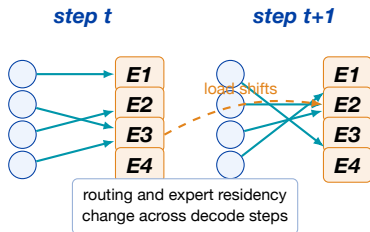
Example

A university assistant may answer 100 different questions using the same handbook. Blindly recomputing the handbook prefix wastes prefill work.

What changes with MoE?

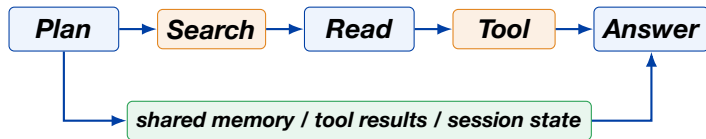
Each token may activate different experts, and the expert load can change from token to token.

- Expert load imbalance.
- Expert parallel communication.
- Expert offloading and residency.
- Dynamic expert-to-device mapping.



Challenge: dynamic expert mapping conflicts with stable graph replay requirements.

Agents: one request becomes a workflow



- Agent requests create intermediate state across multiple steps.
- Scheduling, caching, and isolation now span a workflow, not a single sequence.

Many reuse mechanisms, one selection problem

Selection space

APC, prompt cache, CacheBlend, LMCache, and other mechanisms may all help, but each has different cost and applicability.

- Timing of reuse attempts.
- Request and segment selection.
- Mechanism selection.
- Detection cost versus recomputation cost.

APC

Prompt Cache

CacheBlend

LMCache

Cost-based selector

Modern workloads change the control surface

Workload	New pressure	Control point
Long context RAG	Heavy prefill and large KV Shared but imperfect context overlap	Prefix reuse, paging, admission Segment matching and validation
MoE	Dynamic expert load and residency	Routing, expert placement, graph path
Agents	Multi-step state and tool outputs	Workflow scheduling and isolation

Teaching point

Modern inference systems optimize a moving service workflow, not a single static model invocation.

Platform differences are system differences

- Operator libraries differ.
- Graph execution and compilation behavior differ.
- Interconnect and host-device paths differ.
- Profiling and debugging tools differ.
- Default precision and fallback behavior differ.

Takeaway

A GPU optimization is a hypothesis, not a guaranteed solution, on NPU/DCU/XPU platforms.

Three levels of adaptation

1. **Can run**: model loading, basic inference, correct output.
2. **Can optimize**: platform-aware kernels, graph paths, memory and communication.
3. **Can verify**: fair baseline, controlled workload, reproducible metadata.

Research usually starts at level 2

Running is necessary, but optimization and verification are where system research begins.

Graph replay wants stability; serving wants dynamics

High-performance execution wants

- Stable shapes
- Stable addresses
- Stable memory layout
- Predictable kernels

Online serving gives

- Variable prompt lengths
- Dynamic batching
- Changing KV pages
- MoE routing changes

This conflict is especially visible on graph-oriented accelerator stacks.

Same FLOPs can produce very different latency

Two kernels each perform 1 GFLOP

Kernel A reads 64 MiB from HBM; Kernel B reads 16 MiB.

$$I_A \approx \frac{10^9}{64 \times 2^{20}} = 14.9 \text{ FLOP/byte}, \quad I_B \approx 59.6 \text{ FLOP/byte}.$$

- B has four times the arithmetic intensity.
- Under a bandwidth bottleneck, B has a much lower memory-time bound.
- FLOPs alone do not determine kernel time or end-to-end time.

A 2x kernel speedup may yield only 1.25x end to end

Profile result

The optimized kernel accounts for 40% of request time; everything else accounts for 60%.

$$T_{\text{new}} = 0.60 + \frac{0.40}{2} = 0.80, \quad \text{end-to-end speedup} = \frac{1}{0.80} = 1.25 \times .$$

- Local speedup answers whether the mechanism worked.
- End-to-end speedup answers whether the user benefited.
- A new profile checks whether the bottleneck moved.

Cross-platform correctness starts with a contract

One FP16 output fragment

GPU: [0.5000, -1.2500, 2.0000, 0.1250]

NPU: [0.5005, -1.2490, 1.9980, 0.1252]

1. Maximum absolute error is 0.0020.
2. Compare it with a pre-declared tolerance for this dtype and operator.
3. Separately test masking, boundary shapes, fallback, and multi-step state updates.

Portability means equivalent semantics and observable behavior, not necessarily bitwise identity.

Measurement checklist

- End-to-end: TTFT, TPOT, P99, goodput.
- Runtime probes: graph hit rate, fallback count, memory allocation failures.
- Hardware probes: device utilization, HBM usage, communication time.
- Stability probes: variance across runs, tail behavior under bursty load.

Cause separation

Is the regression caused by scheduling, memory pressure, graph fallback, communication, or benchmark mismatch?

10-Minute Break

Session 3 resumes with heterogeneous execution, evidence, and open engineering.

Before the next session

Keep the comparison matched: same model, shape, dtype, hardware scope, workload, and metric definition.

Part V. Evidence and Open Engineering Practice

Benchmarking, reproducibility, and vLLM-HUST workflow

Why benchmark evidence is fragile

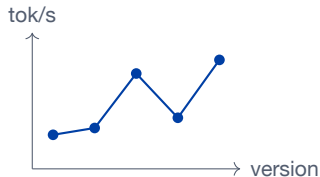
- Same workload name, different token distribution.
- Same model name, different precision or parallelism.
- Same code path, different backend defaults.
- Same throughput, very different tail latency.
- Same benchmark, missing metadata.

A benchmark is an evidence chain: workload, setup, baseline, and metrics.

Minimum metadata for a credible result

<i>Category</i>	Must record
<i>Code</i>	Repository, commit, PR, dependency versions
<i>Model</i>	Model ID, dtype, quantization, parallel strategy
<i>Hardware</i>	Device type, number of cards, driver, runtime stack
<i>Service config</i>	Max length, graph/eager mode, cache/offload settings
<i>Workload</i>	Input/output lengths, concurrency, arrival process, data source
<i>Metrics</i>	TTFT, TPOT, P99, throughput, goodput, memory usage

Example: a misleading performance trend



Consistency checks

- Same model?
- Same input/output length?
- Same dtype?
- Same graph mode?
- Same hardware and driver?

A large jump needs attribution: optimization effect or experiment change.

A controlled A/B result reports a distribution

Hypothesis

A fused kernel lowers operator latency for one fixed shape.

- Hold model, shape, dtype, hardware, version, stream, and timing boundary constant.
- Warm up both paths, then measure each path 30 times.
- Baseline: median 12.4 ms, P95 13.1 ms.
- Fused: median 10.2 ms, P95 10.8 ms.

$$\text{median speedup} = \frac{12.4}{10.2} \approx 1.22 \times .$$

Do not turn 1.22x microbenchmark gain into a system claim

End-to-end measurement on the same workload

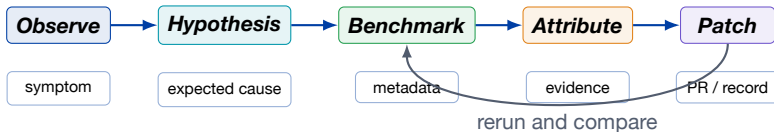
Request latency changes from 100 ms to 96 ms.

$$\text{system speedup} = \frac{100}{96} \approx 1.042 \times .$$

- The operator result is $1.22\times$.
- The user-visible result is about 4.2%.
- Profile data explains the gap by showing the optimized operator's time share.

Report both results, with different labels and scopes.

From observation to evidence



Evidence loop

A performance claim becomes useful only after the baseline, workload, configuration, and code path are recorded and rerun.

Practice target

vLLM-HUST, vLLM-Ascend-HUST, benchmark, and dev-hub repositories organize real inference-system development tasks.

- Start from an issue, CI failure, benchmark gap, or documentation problem.
- Record environment, code path, commands, and failure symptoms.
- Submit a small patch or standardized benchmark record.
- Use CI, review, and reruns to create maintainable evidence.

Hands-on practice path

- Define the system path being changed.
- Build a minimal reproduction.
- Compare against a clear baseline.
- Record metadata and known limitations.
- Produce a patch, PR, benchmark record, or documentation artifact.

Practice goal

Turn a system observation into a reproducible engineering claim.

Trace summary

P99 latency is 1.8 s: queueing 1.1 s, prefill 0.4 s, first decode step 0.2 s, and network 0.1 s.

1. **Fact:** queueing accounts for about 61% of the measured P99 path.
2. **Inference:** admission or capacity pressure may dominate.
3. **Decision:** record candidate sets, budgets, free blocks, and request ages.
4. **Next test:** compare the current policy with an aging-aware policy on a fixed workload.

Closing

Concepts, open questions, and further reading

Six concepts to remember

1. **Attention**: match with QK^T , then mix visible V vectors.
2. **Autoregression**: one selected token unlocks the next generation step.
3. **Prefill and decode**: different shapes, timing, and bottlenecks.
4. **Scheduling and KV**: dynamic batches depend on persistent request state.
5. **Heterogeneous execution**: hardware differences reshape the viable path.
6. **Evidence**: matched workloads and complete metadata bound every claim.

Open research questions

- Global selection among KV reuse mechanisms.
- Boundaries for saving, sharing, or isolating agent intermediate state.
- How can MoE routing coexist with high-performance graph replay?
- Which GPU serving lessons transfer to domestic NPU platforms?
- What benchmark suite best represents bursty, long-tail, mixed workloads?

Recommended progression

1. Hand-calculate one row of causal attention.
2. Trace one token through a Transformer block and sampling step.
3. Draw the prefill/decode lifecycle of one request.
4. Study continuous batching and paged KV state.
5. Read one runtime and reproduce one benchmark with full metadata.

Starting point

A minimal reproducible experiment exposes the real system boundary and prevents accidental claims.

Recommended reading and systems

- Attention Is All You Need. NeurIPS 2017.
- Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI 2022.
- vLLM / PagedAttention: Efficient Memory Management for LLM Serving. SOSP 2023.
- SGLang: Efficient Execution of Structured Language Model Programs.
- LMCache, CacheBlend, Hydragen, and related KV reuse/sharing work.
- vLLM-HUST and vLLM-Ascend-HUST for domestic heterogeneous serving practice.

A useful reading lens: changed control point, introduced state, and demonstrated end-to-end value.

Summary

LLM inference systems build a verifiable service loop across dynamic requests, complex intermediate state, and heterogeneous execution paths.

- Scheduling absorbs dynamic load.
- KV cache determines memory pressure and reuse opportunities.
- Heterogeneous hardware reshapes the execution path.
- Benchmarking decides whether the story is trustworthy.

Thank you!

Questions and Discussion

Shuhao Zhang Huazhong University of Science and Technology